

Final Project Report

Parallel Online Examination System (POES)

Course: Operating Systems
Instructor: Mr Zulfiqar Ali
TA: M.Ahsan Ali Khan
Semester: Spring 2026 (4th Sem)
University: FAST NUCES
Submission Date: 11 May 2026

Group Members:

Name		
Abdullah	24k-0859	Server Architecture, Multithreading, Mutex Synchronizatio
Abdul Majid	24k-0895	Process Management,
Muhammad Hammad	24k-0544	Anti-Cheat Engine, Fork & /proc Fault Tolerance (mmap), Hardware Timers, UI Threading

1. Abstract

The Parallel Online Examination System (POES) is a robust, multithreaded client-server application built entirely in C for the Linux operating system. The primary motivation was to go beyond theoretical learning and directly implement the core concepts of an Operating Systems course into a single, cohesive, working project.

The system enables multiple students to connect to a secure exam server simultaneously over TCP/IP sockets. The server handles each student on an independent POSIX thread, protecting shared data using Mutex locks to eliminate race conditions. The client features an OS-Level Anti-Cheat scanner that uses the fork() system call to spawn a hidden monitoring process, which scans the Linux /proc virtual filesystem to detect unauthorized applications. Student scores are backed up to disk in real-time using Memory-Mapped Files (mmap) for fault tolerance, and an 80-second exam timer is enforced using OS hardware interrupts (SIGALRM). A live countdown timer runs asynchronously in a background thread, updating the terminal UI without blocking student input.

The resulting system demonstrates practical mastery of: IPC Sockets, POSIX Threads, Mutex Synchronization, Process Forking, Signal Handling, Memory-Mapped I/O, and Concurrent UI design.

2. Introduction

In the era of digital education, the ability to run secure, concurrent, and fault-tolerant online examinations is a critical requirement. Traditional sequential programs handle one user at a time; if a second student connects

while the first is being served, they must wait. This is entirely unacceptable in a real-world examination scenario involving an entire class of students.

Operating systems provide the mechanism for true parallelism -- the ability for a single application to manage multiple tasks at the exact same instant. POES exploits this machinery directly, using raw POSIX system calls rather than high-level abstractions, to build a system that:

- Handles multiple students concurrently via kernel-scheduled threads.
- Synchronizes shared data using hardware-level mutex locks.
- Monitors the student's running processes at the OS memory level.
- Persists data to the hard drive through direct memory page mapping.
- Enforces time limits through hardware timer interrupts.

This report documents the complete design, implementation, and testing of each of these systems.

3. Problem Statement

Standard sequential server implementations create a blocking pipeline: one student connects, the server handles them until completion, then the next student is allowed in. This is a fundamental scalability and performance failure.

Additional challenges include:

- Academic Integrity: Students may open browsers or other resources during the exam.
- Data Loss: If the server crashes mid-exam, all score data may be lost.
- Time Enforcement: Without a hard time limit, students can take unlimited time.
- Fairness: If all students receive questions in the same fixed order, the earlier students can leak the sequence to those who have not yet started.

POES was designed to solve each of these problems using direct OS-level mechanisms.

4. Project Objectives

1. Build a TCP Client-Server architecture for networked examinations.
 2. Implement POSIX Threads on the server to handle multiple concurrent clients.
 3. Use Mutex Locks to protect shared memory from Race Conditions.
 4. Implement an OS-Level Anti-Cheat system using `fork()` and `/proc` scanning.
 5. Persist exam scores to disk using Memory-Mapped Files (`mmap`) for zero-overhead fault tolerance.
 6. Enforce an 80-second time limit using Linux hardware timer interrupts (`SIGALRM`).
 7. Display a live countdown timer using a background POSIX thread and ANSI terminal escape sequences.
 8. Randomize the question order for each individual student to prevent cheating.
-

5. System Design and Architecture

5.1. High-Level Architecture

The system operates on a classic Client-Server model over a loopback TCP/IP network.

```
[ Client Process (Student) ]           [ Server Process (Daemon) ]
  ??????????????????????????????    ??????????????????????????????
  ?? Parent Process                    ? Main Thread: Socket Listener    ?
  ??????????????                      - accept() ? pthread_create() ?
  ??                                     ?                                     ?
  ?? Worker Thread (per student): ?    ?                                     ?
  ?                                     ? - Shuffle Questions (Fisher-Yates?
  ?                                     ? - Send/Receive Q&A Loop           ?
  ?                                     ? - Grade answers                  ?
  ? Child Process (fork'd)             ? - Mutex Lock ? Update mmap      ?
  ?   - /proc Scanner                  ? - Mutex Unlock                  ?
  ?                                     ?
  ?   - Sends SIGKILL on cheat ?    ??????????????????????????????
  ??????????????????????????????    ?
                                     ?
                                     ? scores_backup?
                                     ? ?data mmap on
                                     ? disk)
                                     ?
                                     ??????????????????
```

5.2. Project File Structure

```
poes_project/
  ??? Makefile           # Build system for all targets
  ??? test_mutex.sh      # GitHub documentation
  ??? src/               # This document
                        # Concurrency stress-testing script

  ??? server/
    ?   ??? server.c      # Server daemon: Threads, Mutex, mmap
    ?   ??? read_backup.c # Recovery utility for binary mmap file
  ??? client/
    ?   ??? client.c      # Student client: IPC, fork, SIGALRM, UI thread
  ??? common/
    ??? protocol.h        # Shared structs, constants, ANSI color macros
```

5.3. Shared Protocol (protocol.h)

The protocol.h header is included by both server.c and client.c. It defines:

- Network Configuration: Port number (8080), buffer sizes.
- Exam Configuration: NUM_QUESTIONS (10).
- Data Structures: The Question and StudentSession C structs used for both in-memory operations and disk serialization via mmap.
- ANSI Color Codes: Macros like C_RED, C_GREEN, C_CYAN, C_BOLD for the colored terminal UI.

6. Core OS Concepts: Implementation Details

6.1. Inter-Process Communication via TCP Sockets

All data exchange between the server and each student client occurs over TCP/IPv4 network sockets. Sockets are a standard IPC mechanism that allows two separate processes (even on different machines) to communicate through a bidirectional data stream managed by the OS kernel.

Server-side Socket Lifecycle:

```
// 1. Create a socket file descriptor
server_fd = socket(AF_INET, SOCK_STREAM, 0);

// 2. Bind it to a port
bind(server_fd, (struct sockaddr *)&address, sizeof(address));

// 3. Start listening for connections (max 10 in queue)
listen(server_fd, MAX_CLIENTS);

// 4. Block and wait for a student to connect
new_socket = accept(server_fd, ...);
```

Once `accept()` returns with a new socket ID, the server immediately spawns a thread to handle that client, freeing the main loop to accept the next connection without delay.

6.2. Concurrency via POSIX Threads

The server uses POSIX threads (pthreads) to achieve true concurrency. Rather than a "one student at a time" sequential model, the server creates a new kernel-scheduled thread for every connection.

```
// Spawn a new thread immediately after accepting a connection
int *new_sock = malloc(sizeof(int));
*new_sock = new_socket;
pthread_create(&thread_id, NULL, handle_client, (void*)new_sock);
pthread_detach(thread_id);
```

- `pthread_create()` asks the OS kernel to create a new execution context (thread) for the `handle_client` function.
- `pthread_detach()` tells the kernel to automatically clean up the thread's resources when it finishes, without requiring a `pthread_join()`.

Each student thread manages its own: question shuffler, answer buffer, score counter, and socket communication -- all independently from every other running thread.

6.3. Synchronization via Mutex Locks

With many threads running simultaneously, a Race Condition is a critical danger. Consider this scenario:

- Thread A (Student 1) reads total_students_finished = 5.
- Thread B (Student 2) also reads total_students_finished = 5 at the same instant.
- Thread A writes 6, then Thread B also writes 6.
- The scoreboard now shows 6 instead of the correct value of 7. Data is corrupted.

A pthread_mutex_t (Mutual Exclusion Lock) solves this completely. It guarantees only one thread can be inside the "Critical Section" at a time:

```
pthread_mutex_t scoreboard_lock = PTHREAD_MUTEX_INITIALIZER;

// CRITICAL SECTION START
pthread_mutex_lock(&scoreboard_lock);    // Thread is granted exclusive access

int slot = total_students_finished % MAX_CLIENTS;
mapped_scores[slot].score = score;      // Safe modification of shared memory
total_students_finished++;

pthread_mutex_unlock(&scoreboard_lock);  // Lock released, next thread can enter
// CRITICAL SECTION END
```

The server terminal visually logs every lock acquisition and release, providing direct, real-time proof that the OS synchronization mechanism is functioning correctly.

6.4. Process Management: Fork & Anti-Cheat

The fork() system call is one of the most fundamental OS primitives. It instructs the kernel to create an exact clone of the calling process, resulting in two identical processes diverging from that single point.

```
pid_t monitor_pid = fork();

if (monitor_pid == 0) {
    // This block runs ONLY in the Child Process
    monitor_processes(); // Begin scanning /proc forever
    exit(0);
} else {
    // This block runs ONLY in the Parent Process
    // (monitor_pid now holds the child's PID for later use)
}
```

The /proc Virtual Filesystem:

Linux creates a virtual filesystem at /proc that exists entirely in RAM. Each running program gets its own folder named after its Process ID (e.g., /proc/1234/). Inside each folder is a file called comm containing the program's name. The anti-cheat monitor reads this:

```
void monitor_processes() {
    while (1) {
        DIR *dir = opendir("/proc");
        while ((ent = readdir(dir)) != NULL) {
            if (isdigit(ent->d_name[0])) { // Is this folder a PID?
                // ...
            }
        }
    }
}
```

```
    snprintf(path, 512, "/proc/%s/comm", ent->d_name);
    FILE *fp = fopen(path, "r");           // Open the 'comm' file
    fgets(comm, sizeof(comm), fp);         // Read process name
    if (strcmp(comm, "firefox") == 0) { // Forbidden app?
        kill(getppid(), SIGKILL);         // Execute parent process
    }
}
}
sleep(2); // Scan every 2 seconds
}
```

6.5. Memory Management: Memory-Mapped Files

Traditional file saving with `fwrite()` requires an expensive system call that switches the CPU from User Mode to Kernel Mode every single time data is written. For a high-performance server, this is unacceptable overhead.

`mmap()` provides a far more elegant solution: it maps a file on the hard drive directly into the process's virtual memory address space. From the programmer's perspective, you simply read and write normal C variables. The OS kernel's page fault mechanism and virtual memory manager handle the actual disk I/O transparently and asynchronously.

```
// 1. Create (or open) the backup file
int fd = open("scores_backup.dat", O_RDWR | O_CREAT, 0666);

// 2. Size the file to exactly fit our data
ftruncate(fd, MAX_CLIENTS * sizeof(StudentSession));

// 3. Map the file to our pointer ? it's now part of our RAM!
StudentSession *mapped_scores = mmap(NULL,
    MAX_CLIENTS * sizeof(StudentSession),
    PROT_READ | PROT_WRITE,
    MAP_SHARED, // MAP_SHARED means writes go to the REAL file
    fd, 0);

// 4. We can now close the file descriptor; the mapping persists!
close(fd);
```

After this setup, updating `mapped_scores[0].score = 8;` doesn't just update RAM -- the OS kernel guarantees this change is eventually written to `scores_backup.dat` on disk automatically.

6.6. Hardware Timer Interrupts via Signals (SIGALRM)

The exam enforces a strict 80-second time limit. This is not implemented with a simple `sleep()` loop (which would block the program), but with a real Hardware Timer Interrupt.

```
// Register a signal handler function
```

```
signal(SIGALRM, handle_timeout);

// Ask the OS kernel to fire SIGALRM in 80 seconds
alarm(80);
```

When 80 seconds elapse, the Linux kernel itself interrupts the client process -- regardless of what it is currently doing (even waiting for fgets() input) -- and routes execution to:

```
void handle_timeout(int sig) {
    printf(C_BG_RED " ? TIME IS UP! 80 SECONDS ELAPSED ? " C_RESET "\n");
    printf(C_RED "Your exam has been forcefully submitted.\n" C_RESET);
    exit(0);
}
```

7. Key Algorithms

7.1. Fisher-Yates Question Shuffling

To ensure unique question ordering for every student without touching the globally shared question_bank array (which would cause race conditions), each thread creates its own local index array and shuffles it:

```
// Initialize: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
int q_order[NUM_QUESTIONS];
for (int i = 0; i < NUM_QUESTIONS; i++) q_order[i] = i;

// Fisher-Yates Shuffle
srand(time(NULL) + sock); // Unique seed per student
for (int i = NUM_QUESTIONS - 1; i > 0; i--) {
    int j = rand() % (i + 1);
    int temp = q_order[i];
    q_order[i] = q_order[j];
    q_order[j] = temp;
}
// Now use: question_bank[q_order[i]] for each question
```

7.2. Non-Blocking Live Timer UI (Asynchronous Threading)

fgets() blocks the main thread while waiting for keyboard input. To run the countdown timer in parallel, the client spawns a background thread:

```
pthread_t timer_tid;
pthread_create(&timer_tid, NULL, timer_thread, NULL);
```

The timer_thread function wakes up every second and uses ANSI terminal escape sequences to non-destructively update the top-right corner of the terminal without disturbing the user's current input line:

```
// \033[s = Save cursor position
// \033[2;50H = Move cursor to row 2, column 50
// \033[u = Restore cursor to saved position
printf("\033[s\033[2;50H" C_BG_RED " ? TIME LEFT: %02d s " C_RESET "\033[u", time_left);
```

8. Testing and Results

8.1. Functional Testing

Individual functional tests were performed for each component:

Test		
Start server, connect one client	Exam starts, all 10 questions appear	[PASS] Pass
Answer all questions correctly	Score shows 10/10	[PASS] Pass
Open firefox during exam	Anti-cheat fires within 2 seconds	[PASS] Pass
Leave exam idle for 80 seconds	SIGALRM fires, exam force-submits	[PASS] Pass
Run ./read_backup after exam	Student scores printed from .dat file	[PASS] Pass

8.2. Concurrency Stress Test (Mutex Validation)

Using test_mutex.sh, 30 simultaneous client processes were spawned against the server. Each simulated client rapidly fed 10 dummy answers through the exam pipeline concurrently.

Expected Behavior: All 30 threads should complete, the Mutex should prevent any data corruption in the mapped_scores array, and the server should log 30 individual lock/unlock cycles cleanly.

Observed Result: The server successfully handled all 30 concurrent connections. The server terminal logged clean, sequential lock acquisition and release for every thread. Zero data corruption was observed in the final scores_backup.dat file.

9. Conclusion

The Parallel Online Examination System successfully demonstrates practical implementation of six core Operating System concepts within a single, coherent application. By directly invoking Linux system calls rather than relying on high-level frameworks, the team gained deep, hands-on experience with kernel-level process management, thread synchronization, virtual memory, and hardware interrupts.

The project is modular, well-documented, and validated through both functional and concurrency stress testing, confirming that the Mutex implementation fully eliminates race conditions under heavy concurrent load. The anti-cheat engine proves the real-world applicability of the fork() and /proc mechanisms, while the mmap backup system demonstrates a production-grade approach to data persistence.

This project stands as a complete demonstration of the Operating Systems syllabus applied to a real-world problem scenario.
